2.0 Naive Bayes Classifier -Titanic

In [1]:
```
import pandas as pd
```

In [2]:
```
df= pd.read_csv('titanic.csv')
df.head()
```

Out[2]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S

In [3]:
```
df.isnull().sum()
```

Out[3]:
```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age             86
SibSp            0
Parch            0
Ticket           0
Fare             1
Cabin          327
Embarked         0
dtype: int64
```

```
# removing null values of Age// better if we remove later on the updated dataset
```

In [4]:
```
handle = df['Age'].mean()
```

In [5]:
```
handle
```

Out[5]:
```
30.272590361445783
```

In [6]:
```
df.Age=  df.Age.fillna(handle)
```

In [7]:
```
df
```

Out[7]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	0	3	Kelly, Mr. James	male	34.50000	0	0	330911	7.8292	NaN	Q
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.00000	1	0	363272	7.0000	NaN	S
2	894	0	2	Myles, Mr. Thomas Francis	male	62.00000	0	0	240276	9.6875	NaN	Q
3	895	0	3	Wirz, Mr. Albert	male	27.00000	0	0	315154	8.6625	NaN	S
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.00000	1	1	3101298	12.2875	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
413	1305	0	3	Spector, Mr. Woolf	male	30.27259	0	0	A.5. 3236	8.0500	NaN	S
414	1306	1	1	Oliva y Ocana, Dona. Fermina	female	39.00000	0	0	PC 17758	108.9000	C105	C
415	1307	0	3	Saether, Mr. Simon Sivertsen	male	38.50000	0	0	SOTON/O.Q. 3101262	7.2500	NaN	S
416	1308	0	3	Ware, Mr. Frederick	male	30.27259	0	0	359309	8.0500	NaN	S
417	1309	0	3	Peter, Master. Michael J	male	30.27259	1	1	2668	22.3583	NaN	C

418 rows × 12 columns

Assumption: Make a naive assumption that features such as male,class,age,cabin,fare, etc. are independant of each other

In [8]:
```
df.drop(['PassengerId','Name','SibSp','Parch','Ticket','Cabin','Embarked'],axis ='columns',inplace= True
```

In [9]:
```
df.head()
```

Out[9]:

	Survived	Pclass	Sex	Age	Fare
0	0	3	male	34.5	7.8292
1	1	3	female	47.0	7.0000
2	0	2	male	62.0	9.6875
3	0	3	male	27.0	8.6625
4	1	3	female	22.0	12.2875

1 - survived 0 - not survived , At the end we will work on final obtained from inputs and targer for train_test_split()

In [16]:
```
# 1 - survived 0 - not survived
inputs= df.drop('Survived', axis = 'columns')
target = df.Survived
```

Encoding Sex

In [10]:
```
dummies = pd.get_dummies(df['Sex'])
```

In [12]:
```
dummies = dummies.astype(int)
# will display o and 1 instead of True and False
```

In [13]:
```
dummies
```

Out[13]:

	female	male
0	0	1
1	1	0
2	0	1
3	0	1
4	1	0
...	...	...
413	0	1
414	1	0
415	0	1
416	0	1
417	0	1

418 rows × 2 columns

In [ ]:
```
#inputs = inputs.drop('Sex','male' axis='columns')
```

In [14]:
```
df
```

Out[14]:

	Survived	Pclass	Sex	Age	Fare
0	0	3	male	34.50000	7.8292
1	1	3	female	47.00000	7.0000
2	0	2	male	62.00000	9.6875
3	0	3	male	27.00000	8.6625
4	1	3	female	22.00000	12.2875
...	...	...	...	...	...
413	0	3	male	30.27259	8.0500
414	1	1	female	39.00000	108.9000
415	0	3	male	38.50000	7.2500
416	0	3	male	30.27259	8.0500
417	0	3	male	30.27259	22.3583

418 rows × 5 columns

In [17]:
```
inputs
```

Out[17]:

	Pclass	Sex	Age	Fare
0	3	male	34.50000	7.8292
1	3	female	47.00000	7.0000
2	2	male	62.00000	9.6875
3	3	male	27.00000	8.6625
4	3	female	22.00000	12.2875
...	...	...	...	...
413	3	male	30.27259	8.0500
414	1	female	39.00000	108.9000
415	3	male	38.50000	7.2500
416	3	male	30.27259	8.0500
417	3	male	30.27259	22.3583

418 rows × 4 columns

In [21]:
```
inputs = inputs.drop(['Sex'] ,axis='columns')
```

In [22]:
```
inputs
```

Out[22]:

	Pclass	Age	Fare
0	3	34.50000	7.8292
1	3	47.00000	7.0000
2	2	62.00000	9.6875
3	3	27.00000	8.6625
4	3	22.00000	12.2875
...	...	...	...
413	3	30.27259	8.0500
414	1	39.00000	108.9000
415	3	38.50000	7.2500
416	3	30.27259	8.0500
417	3	30.27259	22.3583

418 rows × 3 columns

In [23]:
```
#concatening columns with sex column
```

In [24]:
```
df
```

Out[24]:

	Survived	Pclass	Sex	Age	Fare
0	0	3	male	34.50000	7.8292
1	1	3	female	47.00000	7.0000
2	0	2	male	62.00000	9.6875
3	0	3	male	27.00000	8.6625
4	1	3	female	22.00000	12.2875
...	...	...	...	...	...
413	0	3	male	30.27259	8.0500
414	1	1	female	39.00000	108.9000
415	0	3	male	38.50000	7.2500
416	0	3	male	30.27259	8.0500
417	0	3	male	30.27259	22.3583

418 rows × 5 columns

In [25]:
```
inputs
```

Out[25]:

	Pclass	Age	Fare
0	3	34.50000	7.8292
1	3	47.00000	7.0000
2	2	62.00000	9.6875
3	3	27.00000	8.6625
4	3	22.00000	12.2875
...	...	...	...
413	3	30.27259	8.0500
414	1	39.00000	108.9000
415	3	38.50000	7.2500
416	3	30.27259	8.0500
417	3	30.27259	22.3583

418 rows × 3 columns

In [26]:
```
merged = pd.concat([inputs,dummies],axis = 'columns')
```

In [27]:
```
merged
```

Out[27]:

	Pclass	Age	Fare	female	male
0	3	34.50000	7.8292	0	1
1	3	47.00000	7.0000	1	0
2	2	62.00000	9.6875	0	1
3	3	27.00000	8.6625	0	1
4	3	22.00000	12.2875	1	0
...	...	...	...	...	...
413	3	30.27259	8.0500	0	1
414	1	39.00000	108.9000	1	0
415	3	38.50000	7.2500	0	1
416	3	30.27259	8.0500	0	1
417	3	30.27259	22.3583	0	1

418 rows × 5 columns

In [30]:
```
final = merged.drop(['male'],axis ='columns')
```

In [31]:
```
final
```

Out[31]:

	Pclass	Age	Fare	female
0	3	34.50000	7.8292	0
1	3	47.00000	7.0000	1
2	2	62.00000	9.6875	0
3	3	27.00000	8.6625	0
4	3	22.00000	12.2875	1
...	...	...	...	...
413	3	30.27259	8.0500	0
414	1	39.00000	108.9000	1
415	3	38.50000	7.2500	0
416	3	30.27259	8.0500	0
417	3	30.27259	22.3583	0

418 rows × 4 columns

In [34]:
```
final.columns[final.isna().any()]
```

Out[34]:
```
Index(['Fare'], dtype='object')
```

In [39]:
```
null_count_city = df['Fare'].isnull().sum()
print(f"Number of null values in 'Fare' column: {null_count_city}")
```

```
Number of null values in 'Fare' column: 1
```

In [38]:
```
final.Fare[:50]
```

Out[38]:
```
0       7.8292
1       7.0000
2       9.6875
3       8.6625
4      12.2875
5       9.2250
6       7.6292
7      29.0000
8       7.2292
9      24.1500
10      7.8958
11     26.0000
12     82.2667
13     26.0000
14     61.1750
15     27.7208
16     12.3500
17      7.2250
18      7.9250
19      7.2250
20     59.4000
21      3.1708
22     31.6833
23     61.3792
24    262.3750
25     14.5000
26     61.9792
27      7.2250
28     30.5000
29     21.6792
30     26.0000
31     31.5000
32     20.5750
33     23.4500
34     57.7500
35      7.2292
36      8.0500
37      8.6625
38      9.5000
39     56.4958
40     13.4167
41     26.5500
42      7.8500
43     13.0000
44     52.5542
45      7.9250
46     29.7000
47      7.7500
48     76.2917
49     15.9000
Name: Fare, dtype: float64
```

In [40]:
```
final.Fare = final.Fare.fillna(final.Fare.mean())
final.head()
```

Out[40]:

	Pclass	Age	Fare	female
0	3	34.5	7.8292	0
1	3	47.0	7.0000	1
2	2	62.0	9.6875	0
3	3	27.0	8.6625	0
4	3	22.0	12.2875	1

In [41]:
```
from sklearn.model_selection import train_test_split
```

In [43]:
```
xtrain, xtest, ytrain, ytest = train_test_split(final, target, test_size = 0.30, random_state =1)
```

In [44]:
```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB ()
```

In [47]:
```
model.fit(xtrain, ytrain)
```

Out[47]:
```
GaussianNB
GaussianNB()
```

In [49]:
```
model.score(xtest,ytest)
```

Out[49]:
```
1.0
```

In [50]:
```
xtest[:10]
```

Out[50]:

	Pclass	Age	Fare	female
358	3	30.27259	7.7500	0
164	2	41.00000	13.0000	0
17	3	21.00000	7.2250	0
67	1	47.00000	42.4000	0
4	3	22.00000	12.2875	1
377	2	21.00000	11.5000	0
214	3	38.00000	7.7750	1
290	1	30.27259	39.6000	0
381	3	26.00000	7.8792	0
5	3	14.00000	9.2250	0

In [53]:
```
ytest[0:10]
```

Out[53]:
```
358    0
164    0
17     0
67     0
4      1
377    0
214    1
290    0
381    0
5      0
Name: Survived, dtype: int64
```

In [54]:
```
model.predict(xtest[0:10])
```

Out[54]:
```
array([0, 0, 0, 0, 1, 0, 1, 0, 0, 0], dtype=int64)
```

In [55]:
```
model.predict_proba(xtest[:10])
```

Out[55]:
```
array([[1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [0., 1.],
       [1., 0.],
       [0., 1.],
       [1., 0.],
       [1., 0.],
       [1., 0.]])
```

In [62]:
```
#pclass, Age, Fare, Gender

test = [[1,23.000000,113.2750,0]]
#test = xtest
a= model.predict(test)
if a[0] == 0:
      print("Not Survived")
else:
       print(" Survived")
```

```
Not Survived

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:464: UserWarning: X does not have valid feat
ure names, but GaussianNB was fitted with feature names
  warnings.warn(
```

In [ ]:

Calculate value with cross validation

In [63]:
```
from sklearn.model_selection import cross_val_score
```

In [65]:
```
cross_val_score (GaussianNB(),xtrain,ytrain,cv = 5)
```

Out[65]:
```
array([1., 1., 1., 1., 1.])
```

In [ ]: